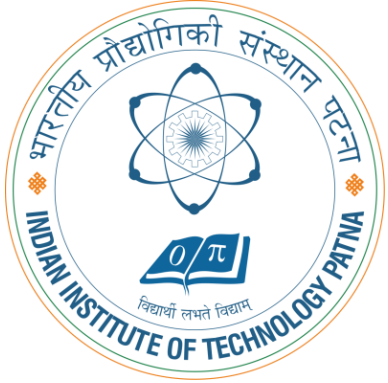


CS5102: FOUNDATIONS OF COMPUTER SYSTEMS

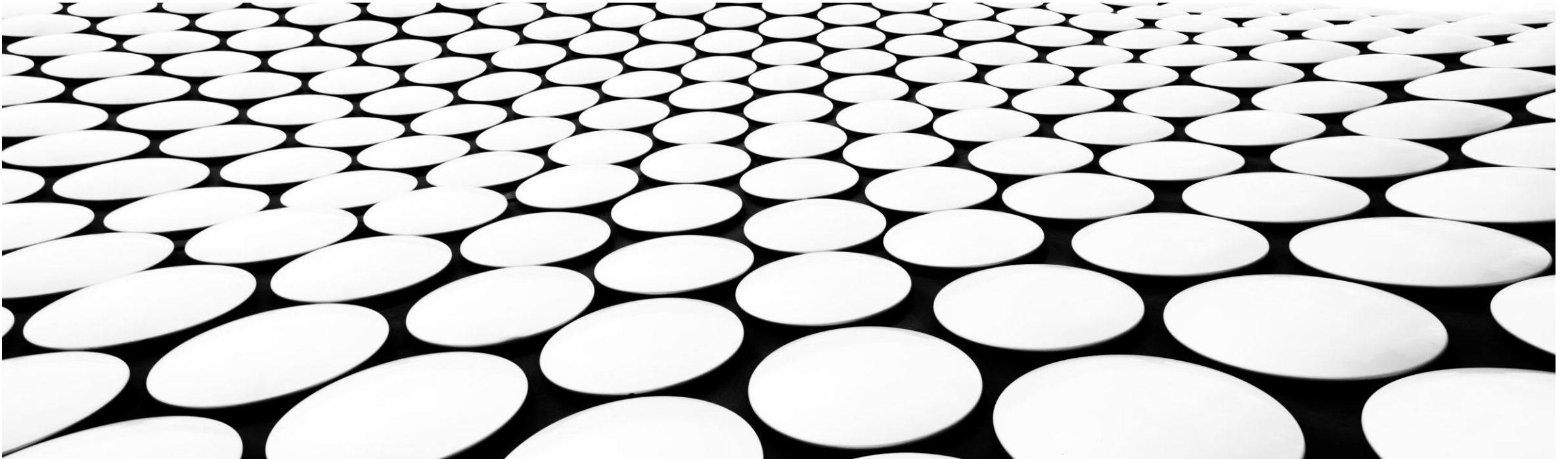


TOPIC-17 PROCESS AND THREAD

DR. ARIJIT ROY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

INDIAN INSTITUTE OF TECHNOLOGY PATNA

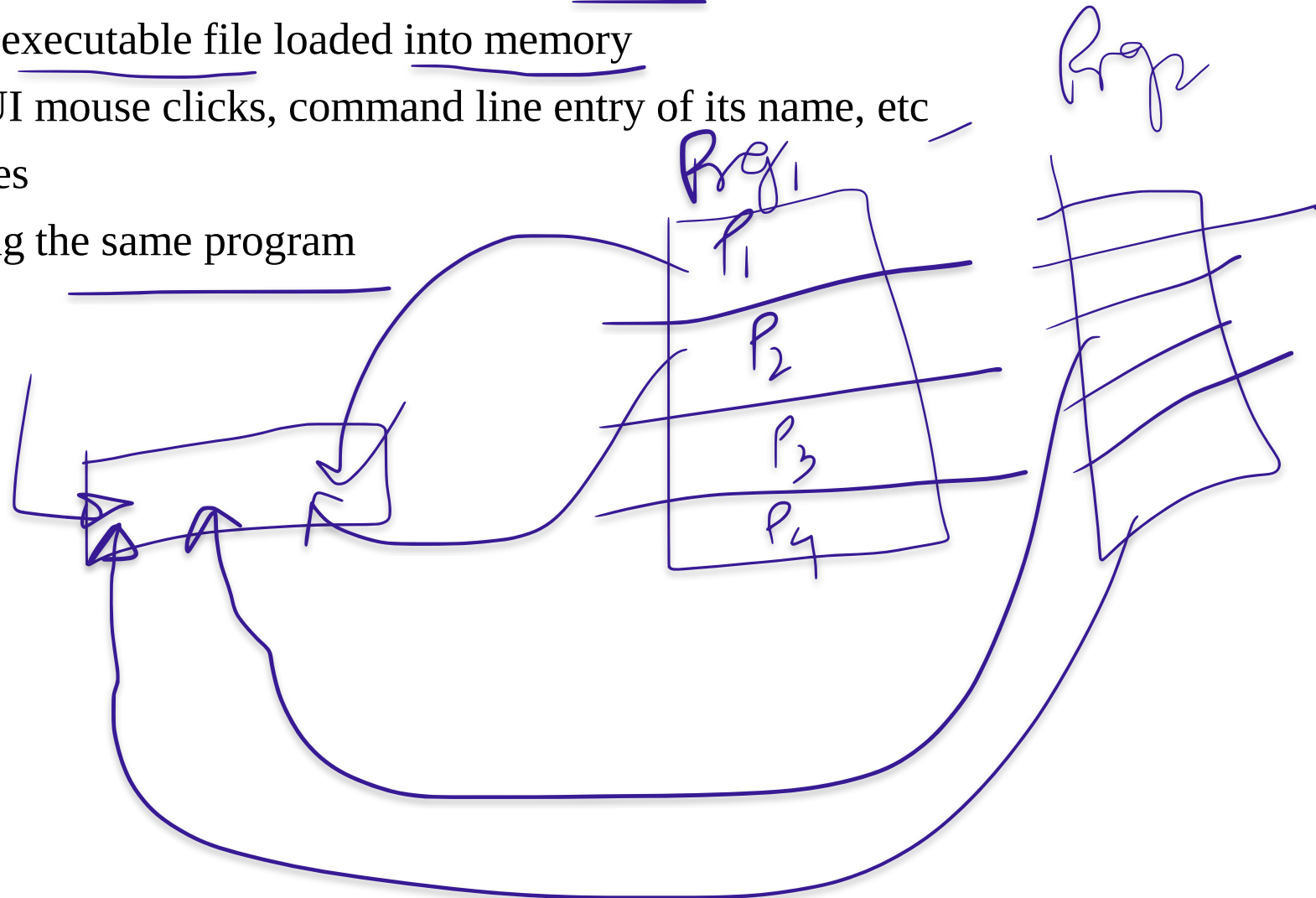


PROCESS CONCEPT

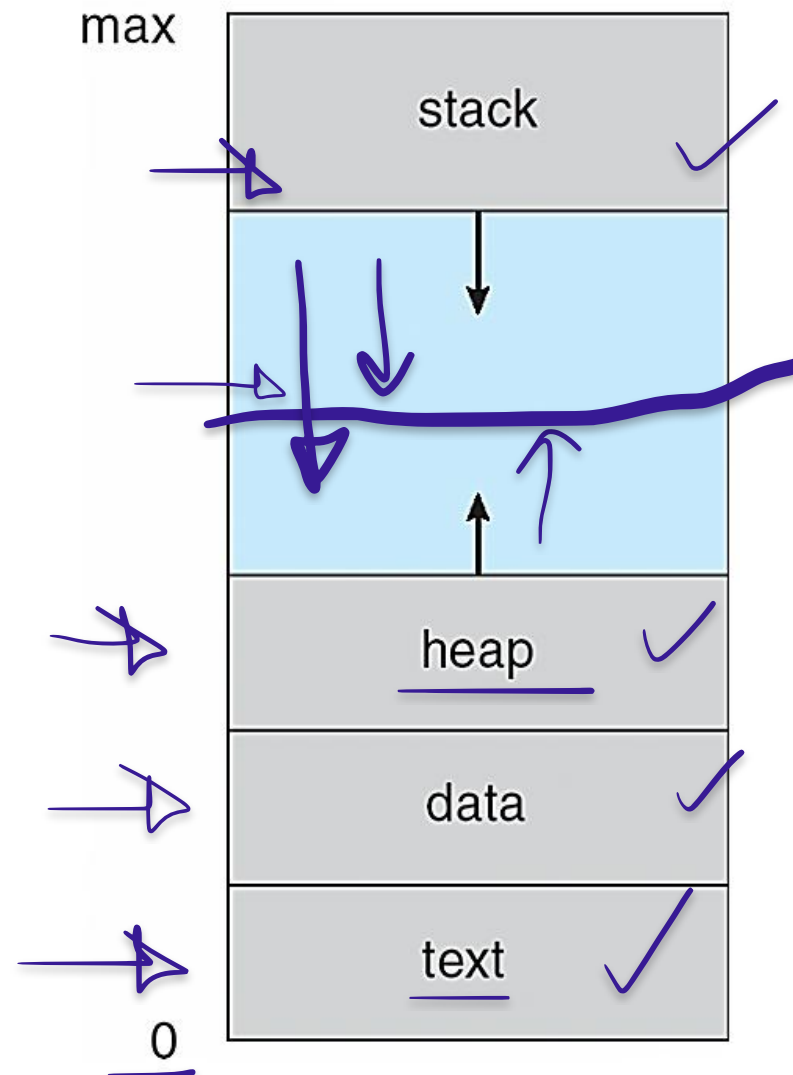
- An operating system executes a variety of programs:
 - Batch system – jobs ✓
 - Time-shared systems – user programs or tasks ✓
- Textbook uses the terms job and process almost interchangeably
- **Process** – a program in execution; process execution must progress in sequential fashion
- Multiple parts
 - The program code, also called text section ✓
 - Current activity including program counter, processor registers
 - Stack containing temporary data
 - Such as *Function parameters, return addresses, local variables*
 - Data section containing global variables
 - Heap containing memory dynamically allocated during run time

PROCESS CONCEPT (CONTD.)

- Program is **passive** entity stored on disk (**executable file**), process is **active**
 - Program becomes process when executable file loaded into memory
- Execution of program started via GUI mouse clicks, command line entry of its name, etc
- One program can be several processes
 - Consider multiple users executing the same program



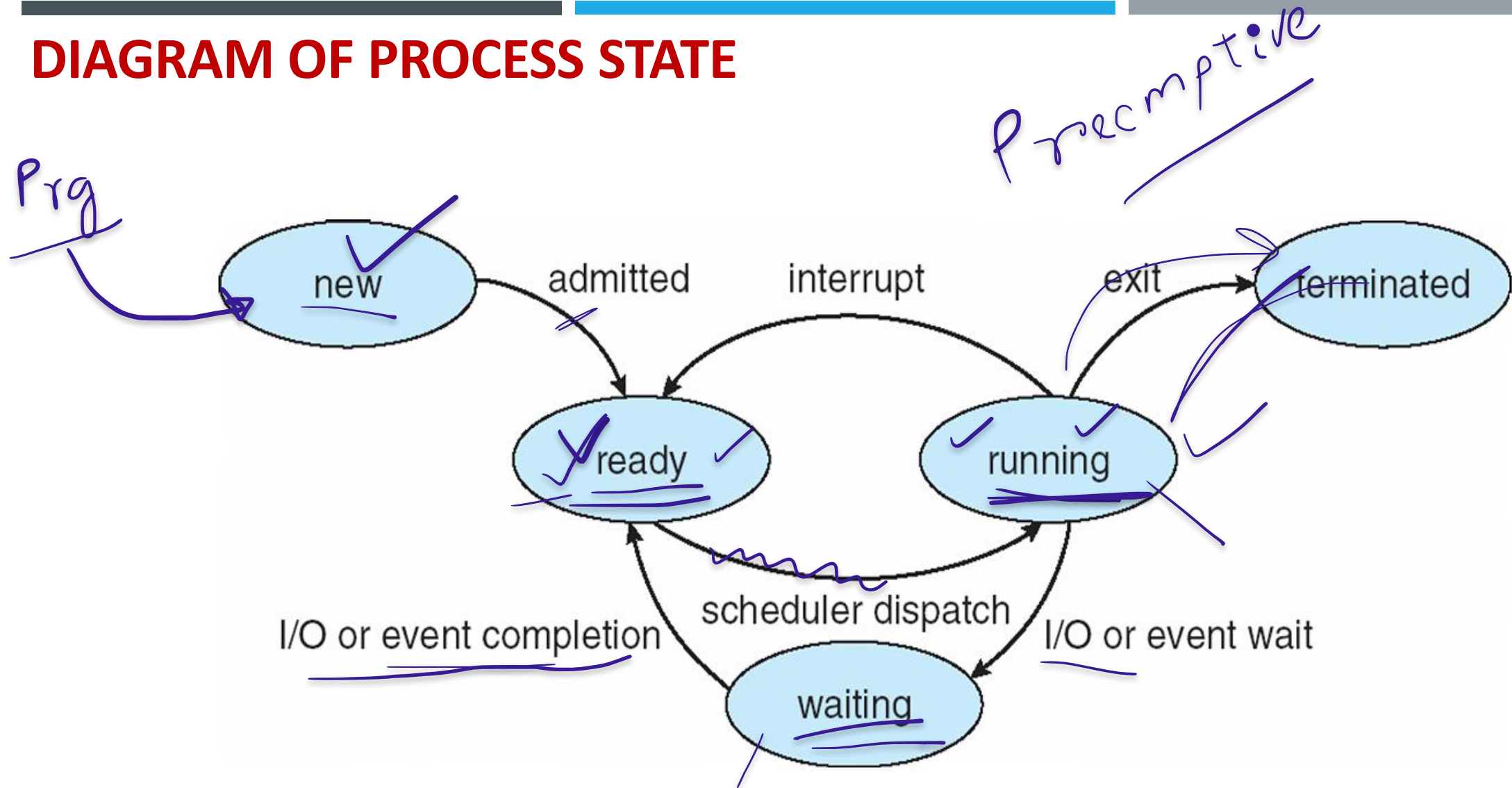
PROCESS IN MEMORY



PROCESS STATE

- As a process executes, it changes **state**
 - ✓ **new**: The process is being created
 - **running**: Instructions are being executed
 - **waiting**: The process is waiting for some event to occur
 - **ready**: The process is waiting to be assigned to a processor
 - **terminated**: The process has finished execution

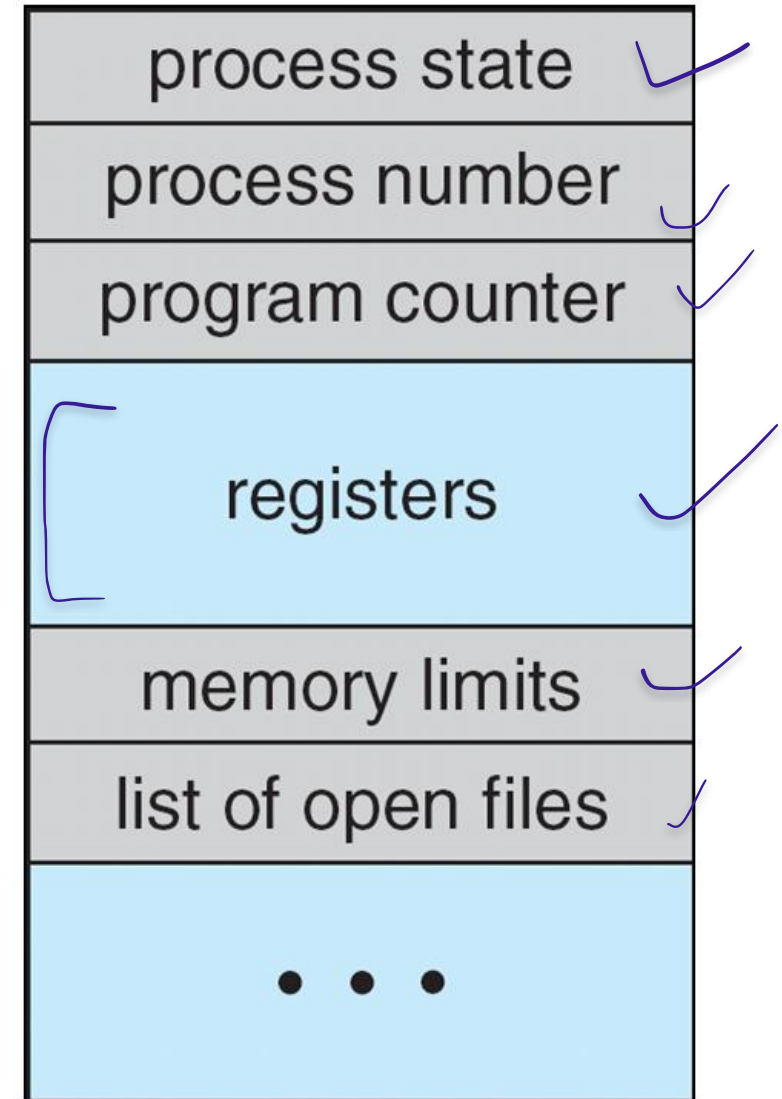
DIAGRAM OF PROCESS STATE



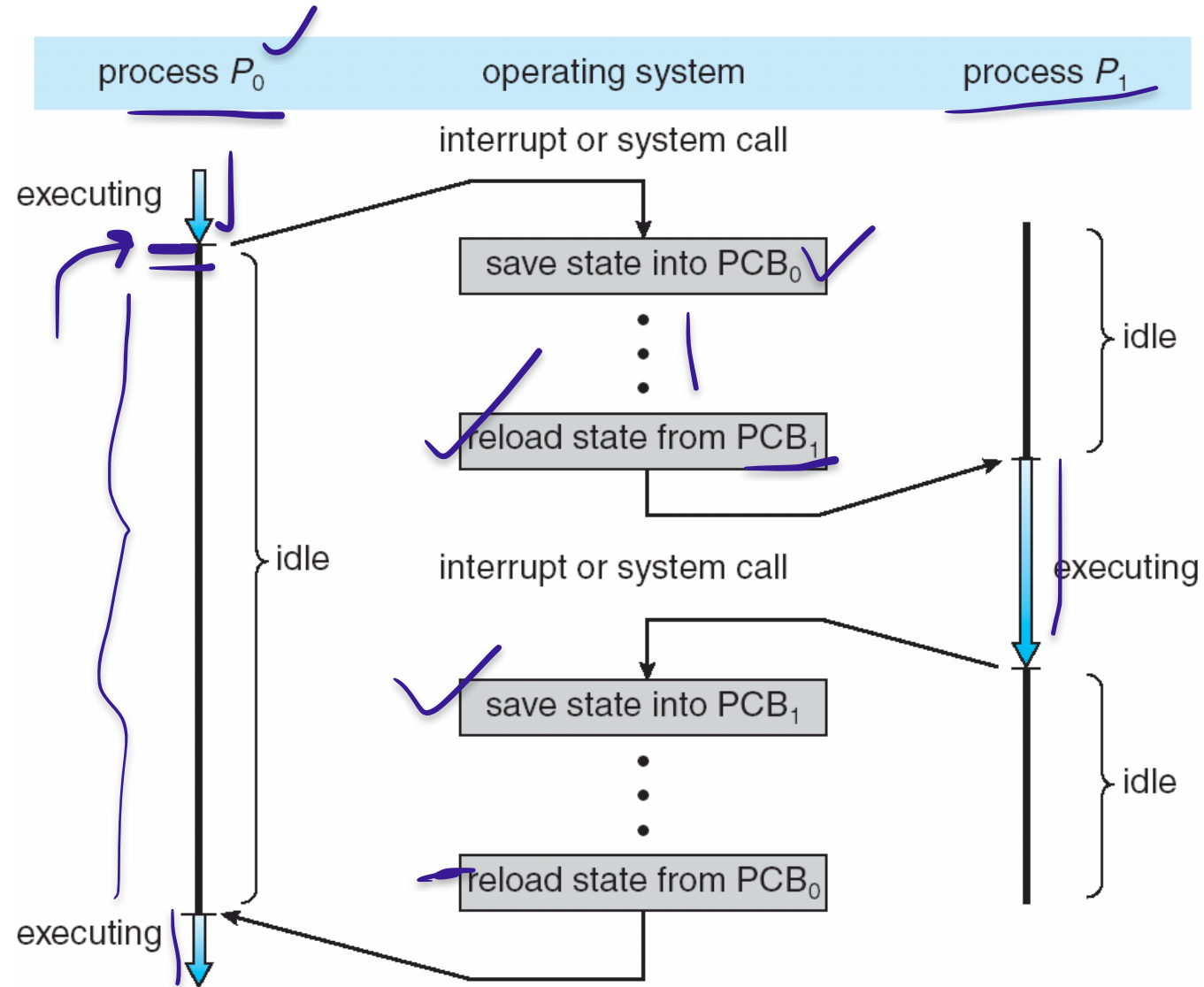
PROCESS CONTROL BLOCK (PCB)

Information associated with each process
(also called **task control block**)

- Process state – running, waiting, etc ✓
- Program counter – location of instruction to next execute ✓
- CPU registers – contents of all process-centric registers – *Accumulators (store intermediate logical or arithmetic data), indexed registers, stack pointer, general purpose registers*
- CPU scheduling information- priorities, scheduling queue pointers
- Memory-management information – memory allocated to the process
- Accounting information – CPU used, clock time elapsed since start, time limits
- I/O status information – I/O devices allocated to process, list of open files



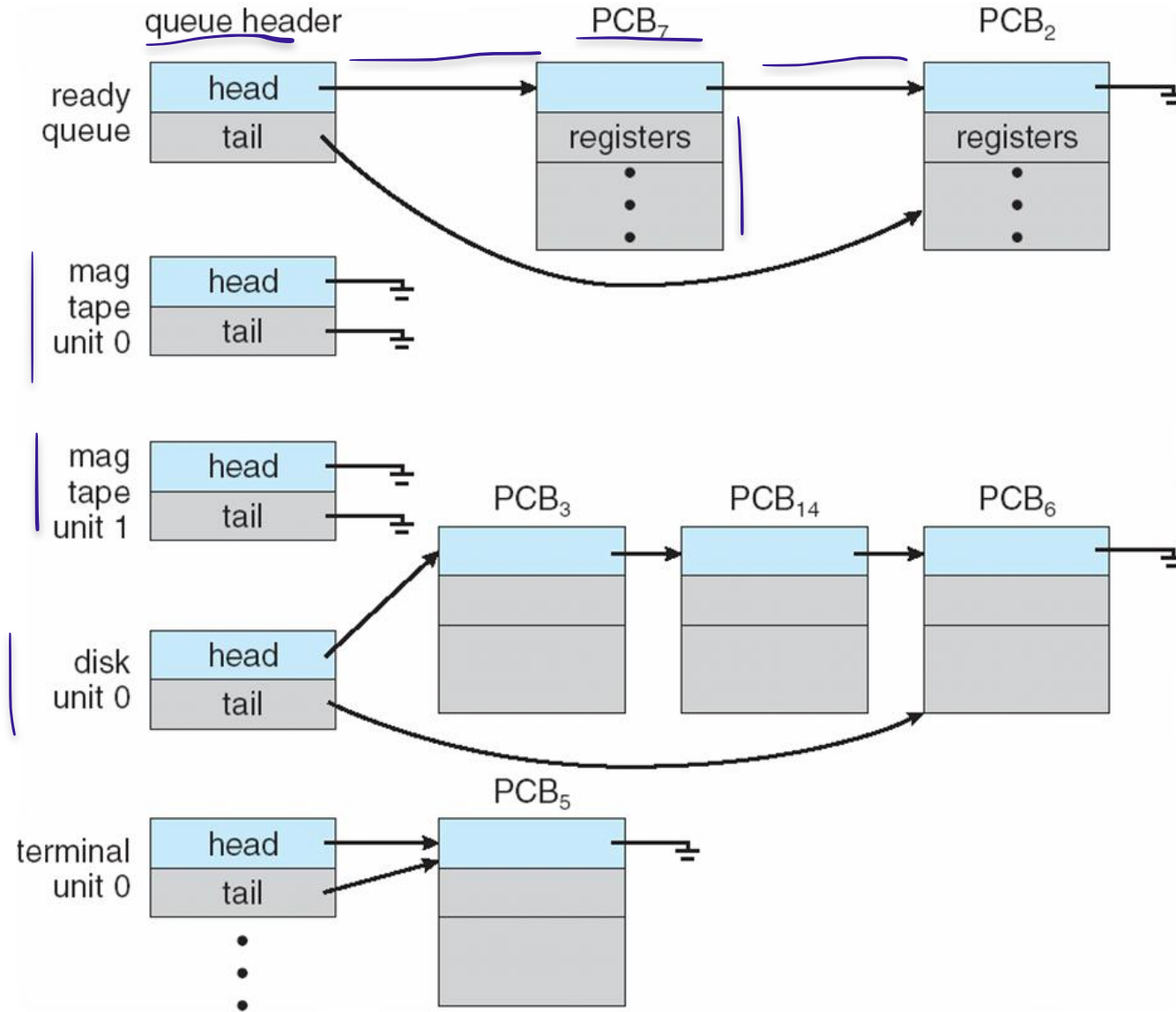
CPU SWITCH FROM PROCESS TO PROCESS



PROCESS SCHEDULING

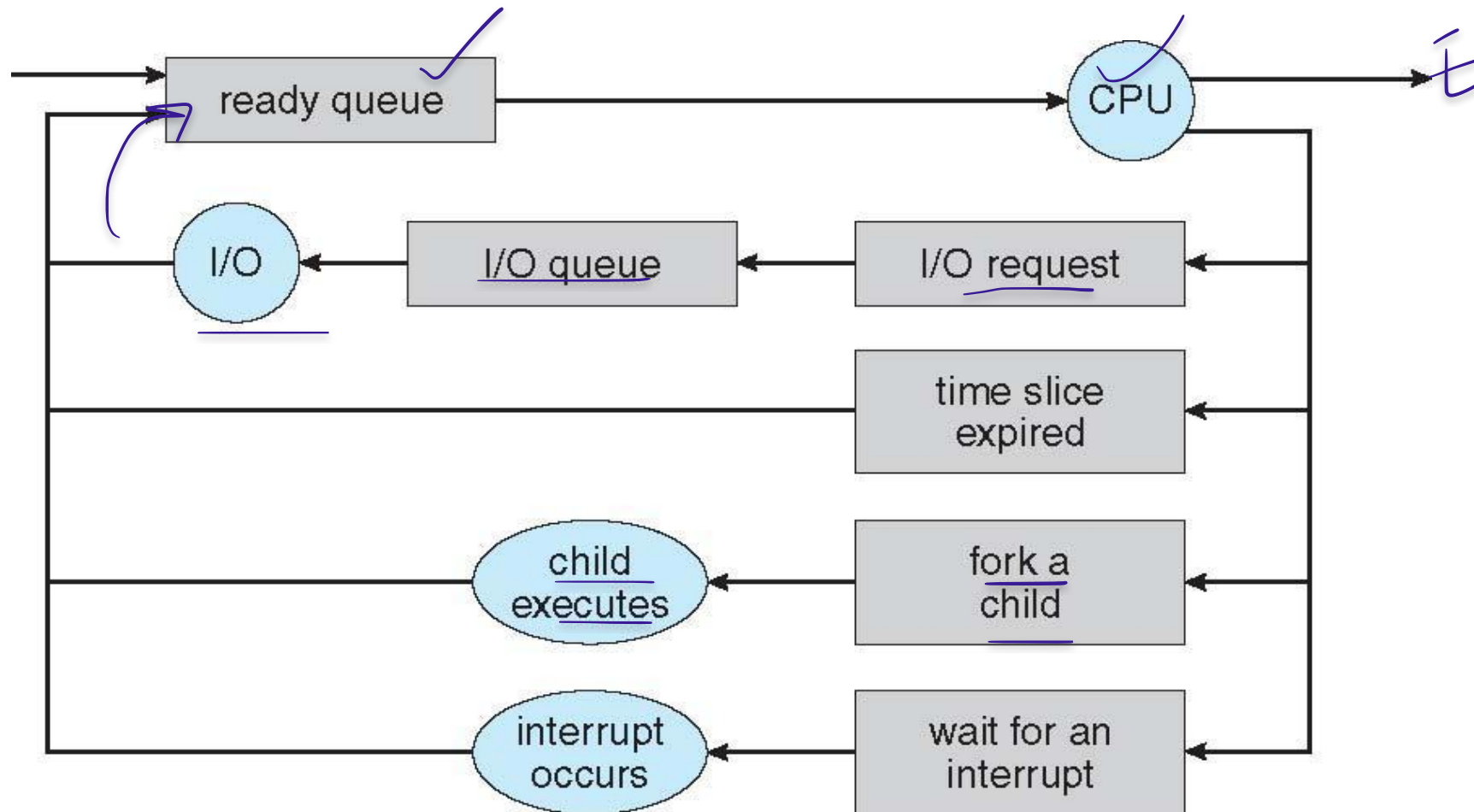
- Maximize CPU use, quickly switch processes onto CPU for time sharing
- **Process scheduler** selects among available processes for next execution on CPU
- Maintains **scheduling queues** of processes
 - ✓ ● **Job queue** – set of all processes in the system
 - ✓ ● **Ready queue** – set of all processes residing in main memory, ready and waiting to execute
 - ✓ ● **Device queues** – set of processes waiting for an I/O device
 - Processes migrate among the various queues

READY QUEUE AND VARIOUS I/O DEVICE QUEUES



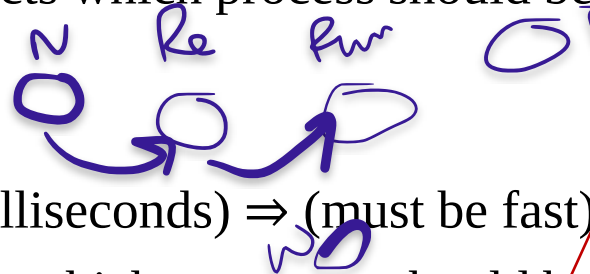
REPRESENTATION OF PROCESS SCHEDULING

- Queueing diagram represents queues, resources, flows



SCHEDULERS

- **Short-term scheduler** (or **CPU scheduler**) – selects which process should be executed next and allocates CPU
 - Sometimes the only scheduler in a system
 - Short-term scheduler is invoked frequently (milliseconds) $\Rightarrow$ (must be fast)
- **Long-term scheduler** (or **job scheduler**) – selects which processes should be brought into the ready queue
 - Long-term scheduler is invoked infrequently (seconds, minutes) $\Rightarrow$ (may be slow)
 - The long-term scheduler controls the **degree of multiprogramming**
- Processes can be described as either:
 - **I/O-bound process** – spends more time doing I/O than computations, many short CPU bursts
 - **CPU-bound process** – spends more time doing computations; few very long CPU bursts
- Long-term scheduler strives for good **process mix**

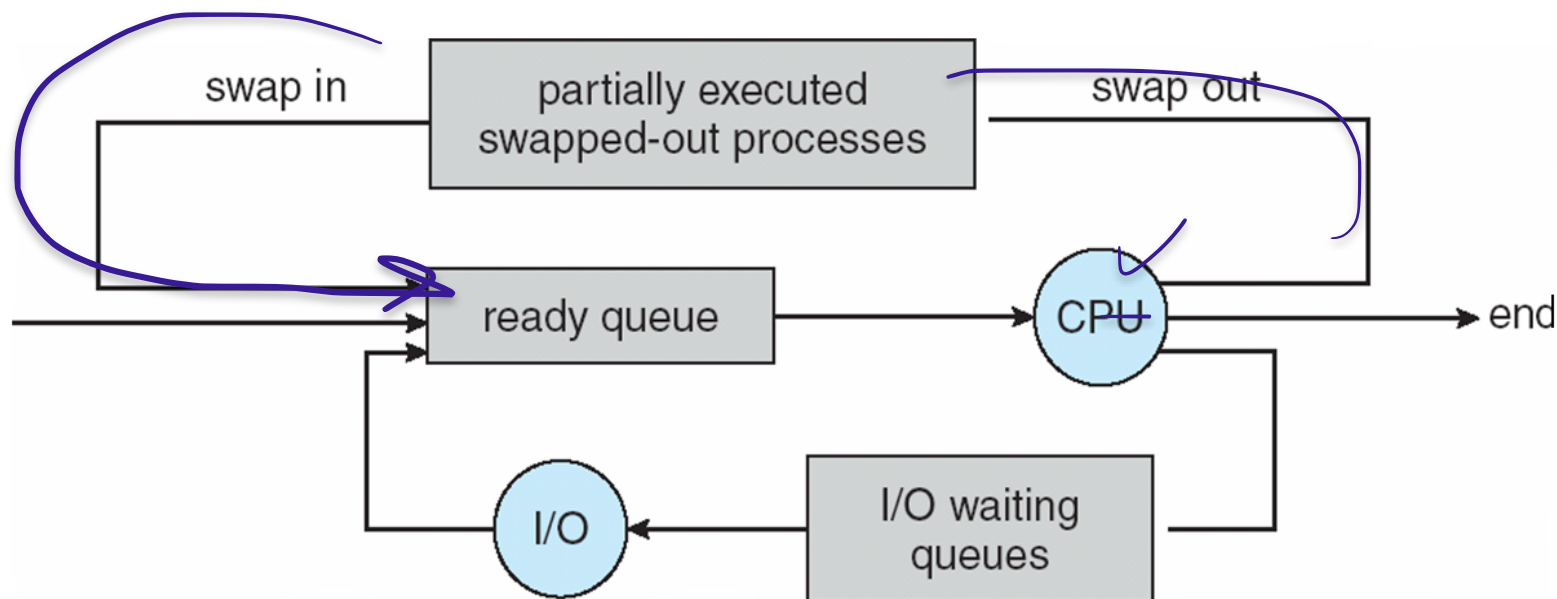


The number of processes in the memory

If the degree of multiprogramming is stable, then the average rate of process creation must be equal to the average departure rate of process leaving the system

ADDITION OF MEDIUM TERM SCHEDULING

- **Medium-term scheduler** can be added if degree of multiple programming needs to decrease
 - Remove process from memory, store on disk, bring back in from disk to continue execution:
swapping



Some systems may have no long-term scheduler or a minimal use of it

e.g., UNIX and Microsoft Windows often have no long-term scheduler

Every process put in to the memory for the short-term scheduler

CONTEXT SWITCH

- When CPU switches to another process, the system must **save the state** of the old process and load the **saved state** for the new process via a **context switch** ✓
- **Context** of a process represented in the PCB
- Context-switch time is overhead; the system does no useful work while switching
 - The more complex the OS >> the longer the context switch
- Time dependent on hardware support
 - Some hardware provides multiple sets of registers per CPU >> multiple contexts loaded at once

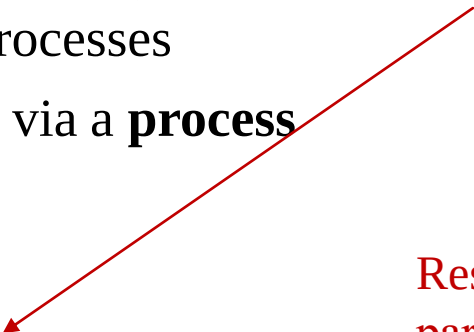
OPERATIONS ON PROCESSES

- System must provide mechanisms for:
 - process creation,
 - process termination,
 - and so on as detailed next

PROCESS CREATION

- **Parent** process create **children** processes, which, in turn create other processes, forming a **tree** of processes
- Generally, process identified and managed via a **process identifier (pid)**
- Resource sharing options
 - Parent and children share all resources
 - Children share subset of parent's resources
 - Parent and child share no resources
- Execution options
 - Parent and children execute concurrently
 - Parent waits until children terminate

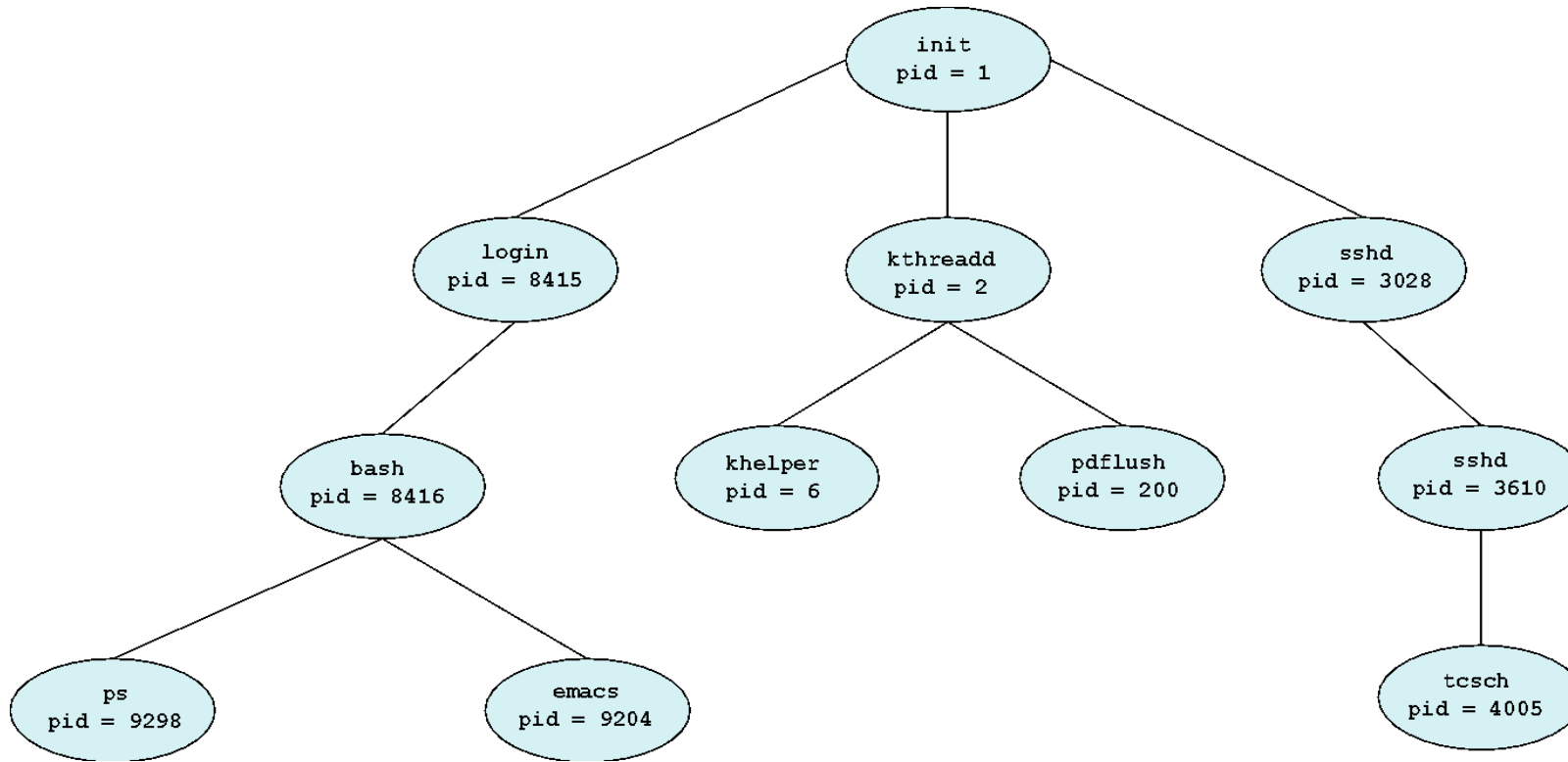
e.g., CPU time, memory, files, I/O devices



Restricting a child process to a subset of parent's resources prevent any process from overloading the system by creating too many child processes



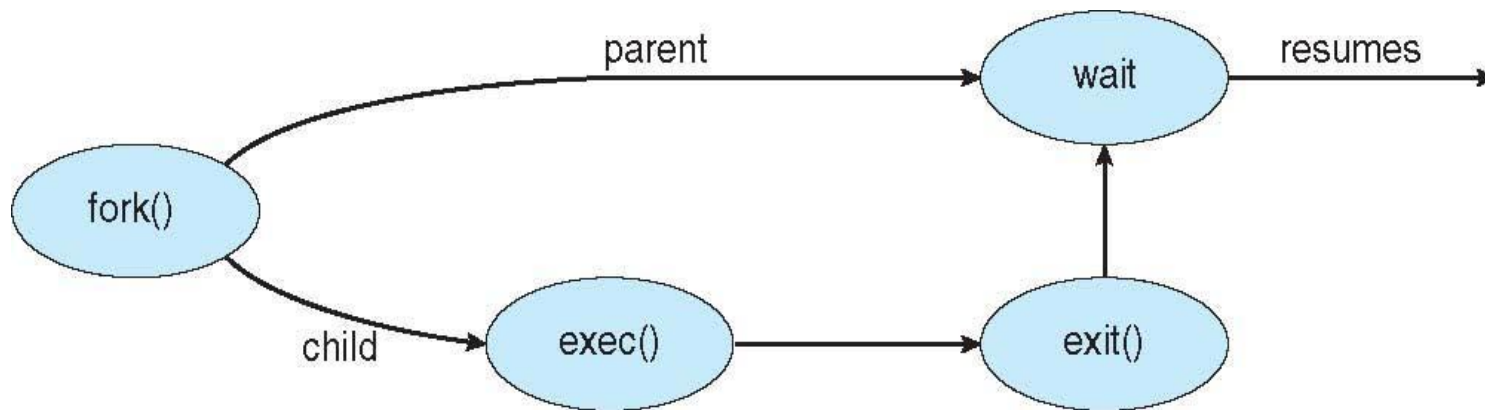
A TREE OF PROCESSES IN LINUX



- Root parent process for all user processes
- Once the system is booted, the *init* process can also create various user processes such as web or print server, ssh server, etc.
- Two child processes – *kthreadd* and *sshd*
- *kthreadd* is responsible for creating additional processes that perform task on behalf of the kernel.
- *sshd* process is responsible for managing the clients that connect to the system by using ssh (secure shell)

PROCESS CREATION (CONT.)

- Address space
 - Child duplicate of parent
 - Child has a program loaded into it
- UNIX examples
 - **fork()** system call creates new process
 - **exec()** system call used after a **fork()** to replace the process' memory space with a new program



C PROGRAM FORKING SEPARATE PROCESS

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0) { /* child process */
        execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
        /* parent will wait for the child to complete */
        wait(NULL);
        printf("Child Complete");
    }

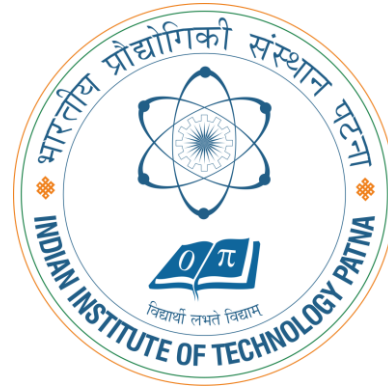
    return 0;
}
```

PROCESS TERMINATION

- Process executes last statement and then asks the operating system to delete it using the **exit()** system call.
 - Returns status data from child to parent (via **wait()**)
 - Process' resources are deallocated by operating system
- Parent may terminate the execution of children processes using the **abort()** system call. Some reasons for doing so:
 - Child has exceeded allocated resources
 - Task assigned to child is no longer required
 - The parent is exiting and the operating systems does not allow a child to continue if its parent terminates

PROCESS TERMINATION

- Some operating systems do not allow child to exist if its parent has terminated. If a process terminates, then all its children must also be terminated.
 - **cascading termination**
 - All children, grandchildren, etc. are terminated.
 - The termination is initiated by the operating system.
- The parent process may wait for termination of a child process by using the **wait()** system call. The call returns status information and the pid of the terminated process
pid = wait(&status);
- Terminated, but whose parent did not invoke **wait()** process is a **zombie**
- If parent terminated without invoking **wait** , process is an **orphan**



THANK YOU!